ChatGPT

Seoul Open Data APIs for Station Areas and Transit Analysis

1. Station-Area Population Composition (2000–2025)

To analyze population around **Seoul Metro Line 1 stations (500m radius)** over 2000–2025, we can leverage both Seoul's open data and Statistics Korea's APIs.

- **Seoul Open Data (Administrative Stats)**: Seoul Open Data Plaza provides registered population statistics by administrative units (dong/gu) with breakdowns by age, household counts, etc. For example, the **주민등록인구 (연령별/동별) 통계** dataset gives quarterly resident population by 5-year age groups for each district [1] [2]. Similar datasets cover household counts and possibly housing types by dong [3]. The Open API base URL format is:

  ```
  http://openapi.seoul.go.kr:8088/{API_KEY}/json/DatasetName/startIndex/
  endIndex/otherParams
  ```

  Authentication: You need a Seoul Data API key (apply on the portal) [4] [5]. For example, to get age-distributed population for a given year:

  ```
  .../json/SeoulPopulationByAgeDong/1/1000/2020
  ```

  (The exact API name and parameters depend on the dataset, often year or date.) The JSON response typically contains fields like region code/name, age group counts, total population, etc. You can request records in batches (max 1000 per call) and iterate by adjusting the start/end indices [6] [7]. Parameter support: These APIs allow filtering by year/period and area. However, **they do not directly provide "500m radius around station"** – you'd need to map each station to the containing dong(s) or use GIS techniques to aggregate nearby administrative units.

- **Statistics Korea SGIS API (Spatial Stats)**: The **Statistical Geographic Information Service (SGIS)** provides a more granular approach. It offers population, households, income, etc., at small-area levels (census blocks or grids) and even allows querying by radius around a point [8]. For example, SGIS's thematic population API can return population by age or household for a custom radius (e.g. 500m) around a given coordinate [8]. Authentication: Requires an SGIS API key and OAuth token from Statistics Korea. Basic URL: `https://sgisapi.kostat.go.kr/OpenAPI3/stats/population.json` with POST body including `radius=500` (m) and coordinates, plus year. Response: JSON with fields like `population_total`, `age_0_9`, `age_10_19`, etc., for the radius, or a list of constituent block data. SGIS data is **official and fine-grained**, so it can more accurately capture a 500m circle than approximating with whole dong populations. However, SGIS time coverage might be limited to census years (2000, 2005, 2010, 2015, 2020) and a few annual estimates, whereas Seoul's admin data is updated quarterly up to 2025 [1].

Accuracy & Resolution: Statistics Korea's data (e.g. census-based) is more precise spatially (small units, exact radius) [8], but temporal resolution is coarser (5-year intervals for full census, or annual registry data by dong). Seoul's open data provides **more frequent updates** (annual/quarterly) up to 2025 [9], but at coarser spatial units (dong or gu) not exact station buffers. In practice, one might use SGIS for spatial accuracy and Seoul's data for recent yearly trends, then compare.

Example DataFrame Design:
For each station and year, you could tabulate fields like:

```
Station | Year | Pop_Total | Age0_14 | Age15_64 | Age65+ | Avg_Income | Housing_Type(%apt) | Households
```

This would come from blending sources (SGIS for radius population by age; perhaps Seoul or KOSIS for income and housing if available by area). In Python, you would call the APIs (e.g. via `requests` to SGIS or Seoul Open API) and normalize the JSON into pandas DataFrames. **All APIs return JSON/XML**, easily consumable in Python [6]. The main effort is joining the station location with the population data. Overall, yes – with some geospatial processing, the data can be compiled for analysis in Python.

2. Citizen-Reported Public Order Issues (Complaints)

For **citizen reports of public order problems** (theft, violence, drunk/disorderly conduct), we turn to Seoul's "Smart Complaint" data services:

- **Seoul Smart Complaint Open Data:** The city's **스마트 불편신고** system logs citizen complaints about various issues. An open dataset **"위치별 불편신고 건수 정보"** provides the number of complaints by location [10]. This is updated monthly – for example, an XLS for each month contains entries of complaint counts at specific coordinates or places [11] [12]. Authentication/Access: Currently, these are provided as downloadable files on the data portal (no single API endpoint for raw records). You can download or programmatically fetch the monthly files (e.g. CSV/XLS) for 2022–2025. Each entry likely includes a location identifier (possibly an address, POI name, or lat-long) and the count of complaints. Some complaint types relevant to public disorder (e.g. noise, harassment) might be included if citizens reported them. However, categories like **theft/violence** are usually police matters, so they might be under-reported in this civic system. There is also a **"분야별 신고 현황"** dataset summarizing complaints by issue category (traffic, environment, safety, etc.) [13]. You would need to identify which categories correspond to theft/violence or disorderly conduct (these could fall under a "public safety" or "police" category if present).

- **Data Fields & Parameters:** In the location-based complaint data, expected fields are: `Location (address or GPS)`, `ComplaintCount`. If the data is aggregated monthly, time-of-day isn't directly given. Unfortunately, **time-of-day breakdown is not provided in the open summary**. For analysis by station and hour, one approach is to obtain the raw complaint dataset via an API if available. (Seoul's API list does include complaint queries, but primarily for FAQs and processing status, not real-time feeds of all complaints.) If raw data were accessible, we'd expect fields like `report_id`, `datetime`, `location`, `category`. We could then filter those where category = theft/violence/etc. and spatially join to the nearest station (within some radius). Since only aggregated data is open, an approximation is to assume each "위치별" entry corresponds to a location like a subway station or vicinity. For instance, if a row is "Station X – 5 complaints", one might assign those to that station. This is a limitation: **we cannot get precise hourly incident counts per station from the open data**.

Example DataFrame Design:
You might create a table like:
`Station | Month | Theft_Complaints | Violence_Complaints | Disorder_Complaints | Total_Complaints`
If hour segregation was possible, add columns or a separate dimension for time (e.g. separate DataFrame of hourly counts). In practice, with the given data, you may only have monthly totals by station vicinity. You can load the monthly XLS files into pandas, extract rows related to stations (by matching station names or coordinates), and then sum by category manually (if the data provides categories – e.g. an entry might have subfields for type of issue). **Using Python:** This data is usable by reading the provided files into DataFrames. If an API endpoint becomes available (e.g. a JSON service for complaints), you'd use `requests` to retrieve JSON. But as of now, expect to handle it via file download. Overall, it's feasible to use Python/pandas for this, but **the analysis will be limited by data granularity** (station mapping and lack of hourly detail).

3. Subway Ridership and Congestion by Time of Day

For **Seoul Metro Line 1 ridership and congestion**, Seoul's open APIs are well-developed:

- **Ridership Open API (승하차 인원):** The key API is **"서울시 지하철호선별 역별 승하차 인원 정보"**, which provides daily passenger counts by station and line [14]. The dataset is updated daily (with a 3-day lag) and includes all lines 1–9 (Seoul Metro, Korail-operated sections, etc.). Access: Use the Seoul Open API with your key. The endpoint is:

  ```
  http://openapi.seoul.go.kr:8088/{API_KEY}/json/CardSubwayStatsNew/
  1/1000/{date}
  ```

  Here `CardSubwayStatsNew` is the dataset name [15], and `{date}` is in YYYYMMDD format [16]. The response JSON contains an array `row` under `CardSubwayStatsNew`. Each element has fields like: `LINE_NUM` (line number or name), `SUB_STA_NM` (station name), `RIDE_PASGR_NUM` (boarding count) and `ALIGHT_PASGR_NUM` (alighting count) for that date [17] [18]. You can specify any date from 2015 to 2025; for multiple days, loop over dates (the API returns up to 1000 station entries per call [19] which covers all stations for one day). This API does not break down by hour – it's total daily ridership per station. However, there is another dataset for time-of-day breakdown.

- **Time-of-Day Breakdown: "서울시 지하철 호선별 역별** 시간대별 **승하차 인원"** provides how many passengers boarded/alighted during each time interval of the day [20]. Data is updated monthly (each month's data added by the 5th of next month) [21]. While provided as files on the portal (CSV by month or year), it is also accessible via Open API. The likely API endpoint is similar, e.g.:

  ```
  .../{API_KEY}/json/CardSubwayTime/1/1000/{date}
  ```

  where `{date}` might retrieve a specific day's hourly stats. (The exact API name and usage can be found in Seoul's API list – it's often documented as CardSubwayTime or similar [22].) In the JSON, each record would include station, line, and fields for each hour or time slot (e.g. `LINE_NUM, SUB_STA_NM, FR_CODE, TIME_05_06, ... TIME_23_24, RIDE_PASGR_NUM, ALIGHT_PASGR_NUM` etc). For example, one entry might show that at **Station X (Line1)**, **7–8 AM**

boardings = 5000 and alightings = 2000, etc. Parameters: likely you specify a date or month. If an entire month is requested, the API might return multiple records (one per station per day). In practice, many analysts download the monthly CSV (e.g. "2023_시간대별_승하차인원.csv"). This detailed data enables computing rush-hour vs off-peak usage at each station.

- **Congestion Index API:** Seoul Metro provides a **train congestion dataset** for lines 1–8. The data is a percentage of capacity filled, averaged over 30-minute intervals [23]. It distinguishes weekdays vs weekends, up/down direction, by station. For example, an entry might be: Weekday, Line1, Station "Seoul Station", Up-direction – 7:30am: 150%. A congestion of 150% means 1.5 times the seating capacity of that car (i.e. very crowded). The data is updated quarterly since 2024 [24]. It's available as files (CSV/XLS) and via the public data portal API. On data.go.kr, this is listed as **"서울교통공사_지하철혼잡도정보"** with JSON/XML available [25] [26]. To use it via API, one can either download the CSV or call the converted API. The API base URL (via data.go.kr's proxy) would be something like:

  ```
  https://api.odcloud.kr/api/15071311/v1/uddi:{resource-id}
  ```

  (after an application on the portal). Given the complexity, many just download the file. The **fields** include: `요일구분` (day type: weekday/Sat/Sun), `호선` (line), `역명` (station name), `상하선구분` (up/down direction), and then columns for each 30-min slot (e.g. `06:00-06:30`, `06:30-07:00`, …) with congestion % [27] [28]. No extra parameters – each file covers the whole network for a quarter.

Example DataFrame Design:
You can create one DataFrame for ridership and one for congestion, and merge by station if needed:

- Ridership DF: `Date | Line | Station | Boardings | Alightings | Hour(optional)` – If using time-of-day data, include an "Hour" column for each station-hour entry. Otherwise for daily, one row per station-day.
- Congestion DF: `Line | Station | DayType | TimeInterval | Congestion(%)`.

For instance, a congestion DataFrame might look like:

```
Line  Station       DayType   Time     Congestion
1     Seoul Station Weekday   07:00    152
1     Seoul Station Weekday   07:30    146
…
```

and so on.

Python Usage: These datasets are straightforward to use in Python. The Open API returns JSON which can be parsed with `requests` and pandas (`df = pd.json_normalize(response.json()['CardSubwayStatsNew']['row'])` for ridership [17] [29]). The CSV/XLS files can be read with `pd.read_csv()` or Excel readers. The APIs have high volume (one day's data has ~600 station records) but support pagination if needed [19]. With a valid API key and mindful request pacing (1000 calls/day limit for Seoul API [30]), you can pull multi-year historical data. In summary, **yes, the ridership and congestion data are readily usable in Python** – they are well-structured and updated, enabling detailed analysis of station-level demand and crowding.

Sources: The information above is based on official Seoul Open Data descriptions and examples [6] [23], as well as Statistics Korea's SGIS documentation [8]. All these data can be combined in Python to assess feasibility of analysis – and indeed they are commonly used for urban data science projects.

---

[1] [2] [3] [9] 서울 열린데이터광장
https://data.seoul.go.kr/dataList/10718/S/2/datasetView.do

[4] [5] [30] 서울 열린데이터광장
https://data.seoul.go.kr/together/guide/useGuide.do

[6] [7] [15] [16] [17] [18] [19] [29] [파이썬] 서울 공공데이터 포털 Open API를 이용해서 지하철 승차인원수 구하기 | Exobrain
https://gunn.kim/posts/%ED%8C%8C%EC%9D%B4%EC%8D%AC-%EC%84%9C%EC%9A%B8-%EA%B3%B5%EA%B3%B5%EB%8D%B0%EC%9D%B4%ED%84%B0-%ED%8F%AC%ED%84%B8-open-api%EB%A5%BC-%EC%9D%B4%EC%9A%A9%ED%95%B4%EC%84%9C-%EC%A7%80%ED%95%98%EC%B2%A0-%EC%8A%B9%EC%B0%A8%EC%9D%B8%EC%9B%90%EC%88%98-%EA%B5%AC%ED%95%98%EA%B8%B0/

[8] 대화형 통계지도 - 통계지리정보서비스 - 통계청
https://sgis.kostat.go.kr/view/map/interactiveMap

[10] [11] [12] 서울 열린데이터광장
https://data.seoul.go.kr/dataList/OA-12053/S/1/datasetView.do?tab=A

[13] 서울 열린데이터광장
https://data.seoul.go.kr/dataList/OA-12051/S/1/datasetView.do?tab=A

[14] 서울 열린데이터광장
https://data.seoul.go.kr/dataList/OA-12914/S/1/datasetView.do

[20] [21] 서울 열린데이터광장
https://data.seoul.go.kr/dataList/OA-12252/S/1/datasetView.do

[22] 서울시 열린데이터광장 Open API 현황
https://data.seoul.go.kr/dataList/OA-2191/S/1/datasetView.do

[23] [24] 서울 열린데이터광장
https://data.seoul.go.kr/dataList/OA-12928/F/1/datasetView.do

[25] [26] [27] [28] 서울교통공사_지하철혼잡도정보_20250331 | 공공데이터포털
https://www.data.go.kr/data/15071311/fileData.do